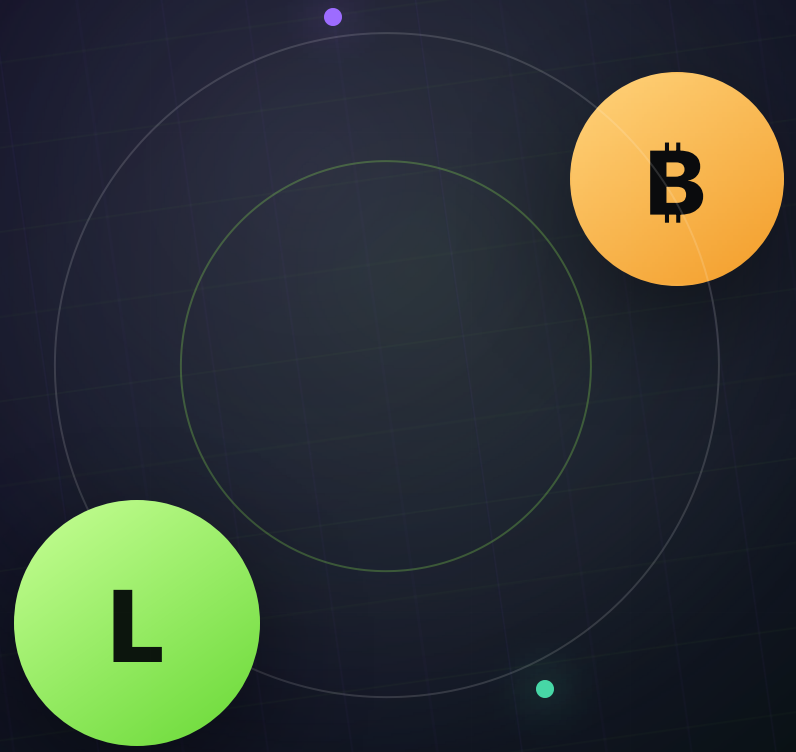
RFP-003 · SUBMISSION PACK

BILATERAL ATOMIC SWAPS

LEZ ⇄ Bitcoin

M1 DESIGN — **M3** BITCOIN — **M6** EXPERIENCE



PRIVATE LOCAL · BITCOIN CORE REGTEST · LEZ v0.2 DEVNET

What this submission covers

M1

Protocol foundation

Roles, threat model, cryptographic primitives, deadlines, escrow custody, and SDK boundaries.

DESIGN + REVIEW PACKET

M3

Bitcoin integration

Taproot adaptor flow, actual-node settlement, ordered refunds, replay safety, and public evidence.

IMPLEMENTATION + EVIDENCE

M6

Basecamp experience

Separate Maker and Taker packages for offers, role-owned actions, progress, history, and proof.

PRODUCT SURFACE + TESTS

DIRECT VIDEO PROOF M3 happy settlement through the branch's current Basecamp BTC journey

Scope and provenance · `docs/submission.md`

M1

Protocol foundations

M1

No third party holds funds or matches orders.

01

Self-custodial

Each role keeps its own keys, state and recovery authority. Bitcoin custody is a 2-of-2 MuSig2 output neither side can spend alone. The LEZ refund instruction takes no signer at all and can only pay the recorded depositor.

No bridge · no custodian · no third-party signer

02

No liquidity pool

Every swap is bilateral: two parties, and the exact amounts fixed in the countersigned agreement before either chain moves. No pooled capital, no LP position, no pricing curve.

No pool to drain · no shared inventory

03

Peer-to-peer offers

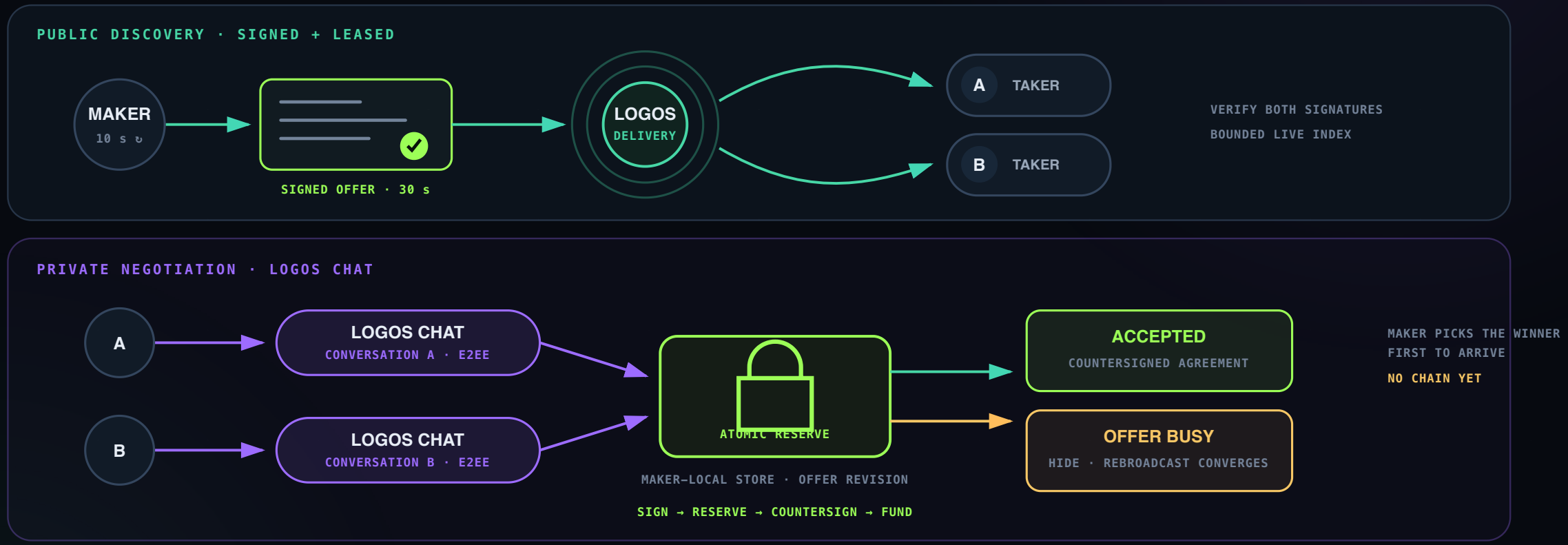
Offers travel as signed envelopes from maker to taker. The taker verifies the maker's signature and the exact terms before consenting to anything. Nothing on that path takes custody or matches orders.

No venue custody · no matching engine

Consequence: recovery never depends on the counterparty's cooperation, on an operator staying online, or on a third party staying solvent. Each side's refund is a transaction it can already build and broadcast itself.

M1 · TRANSPORT BOUNDARY

Broadcast to discover. Chat to negotiate.

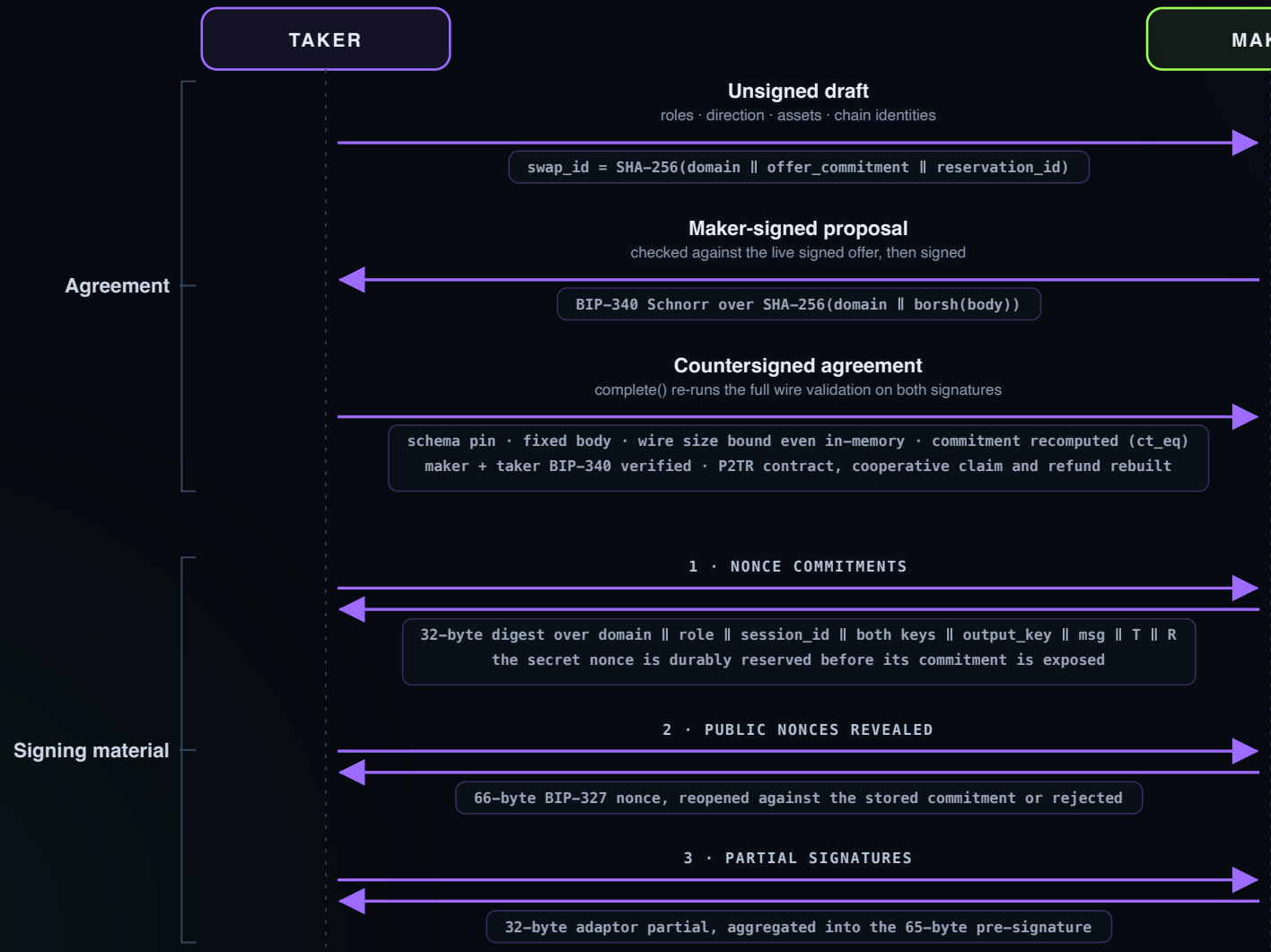


Implementation boundary · ADRs 0210–0211 · topic `/lez-atomic-swaps/1/offers/json`

M1

checkpoint **m1-complete.1**
96b7b22

Nothing is funded until refunds are signed.



What is a nonce?

Every Schnorr signature needs a fresh secret nonce, published as a point R. In MuSig2 both sides contribute one and the two are summed. Seeing the peer's R before choosing your own would let you grind it and forge, so each side publishes a hash of R first and reveals R only once both commitments are held.

Why “signing material”?

None of this is a signature yet. The three rounds produce a pre-signature: a valid signature short by exactly one scalar t. It is safe to hand over, and becomes spendable only when t is published on the other chain.

Recovery ready first

signed deadlines · exact refund paths · a refund needing no counterparty signature

Funds move last

persist before send · canonical observation · bounded authority

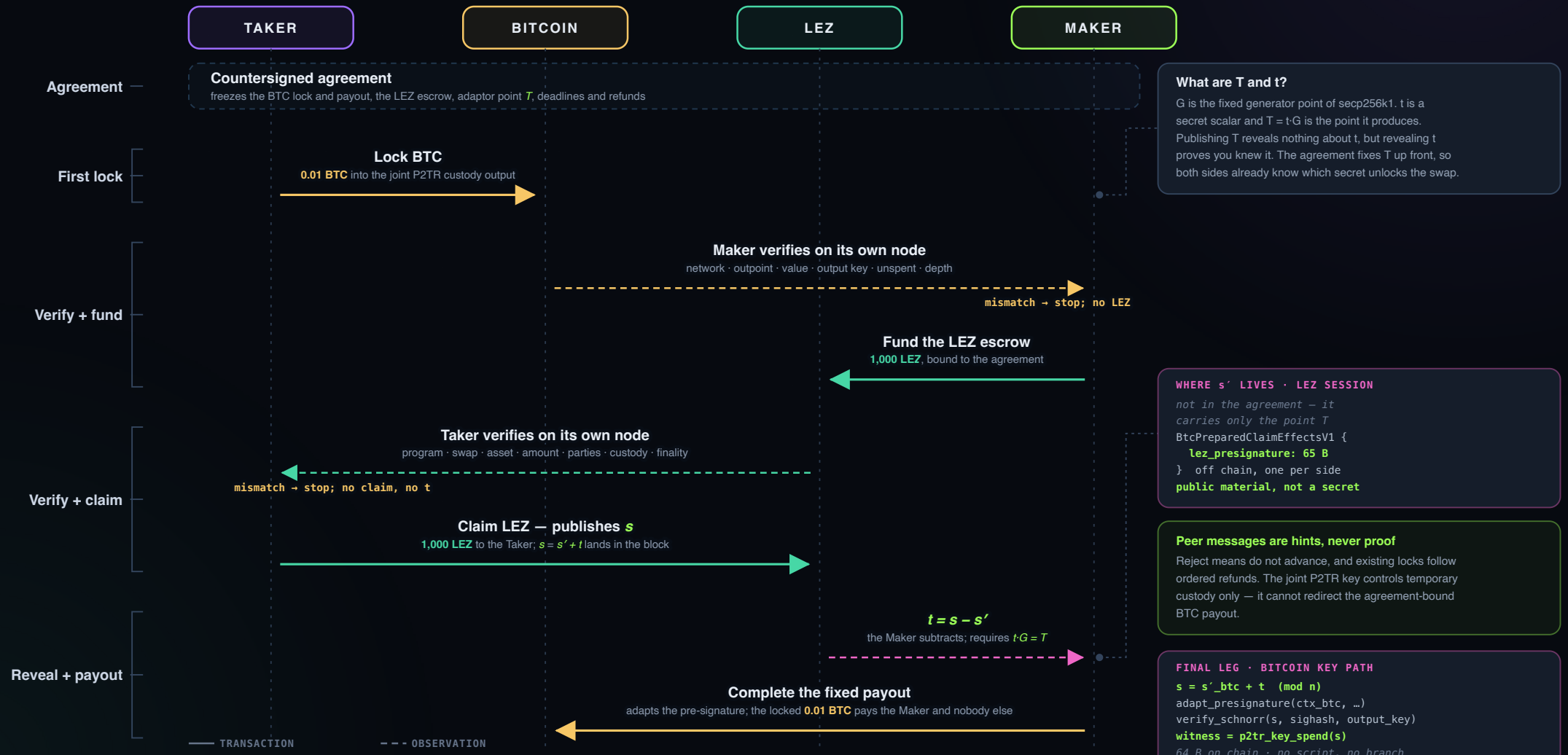
Review entry · `docs/milestone-1/README.md` · 8 design docs · 13 foundation ADRs · 42 traced requirement IDs

M3

LEZ ⇄ Bitcoin leg

M3 · SWAP ALGORITHM · DIRECTION A · LEZ → BTC

Taker sells BTC



What are T and t?

G is the fixed generator point of secp256k1. t is a secret scalar and T = t·G is the point it produces. Publishing T reveals nothing about t, but revealing t proves you knew it. The agreement fixes T up front, so both sides already know which secret unlocks the swap.

WHERE s′ LIVES · LEZ SESSION

```
not in the agreement — it
carries only the point T
BtcPreparedClaimEffectsV1 {
  lez_presignature: 65 B
}  off chain, one per side
public material, not a secret
```

Peer messages are hints, never proof

Reject means do not advance, and existing locks follow ordered refunds. The joint P2TR key controls temporary custody only — it cannot redirect the agreement-bound BTC payout.

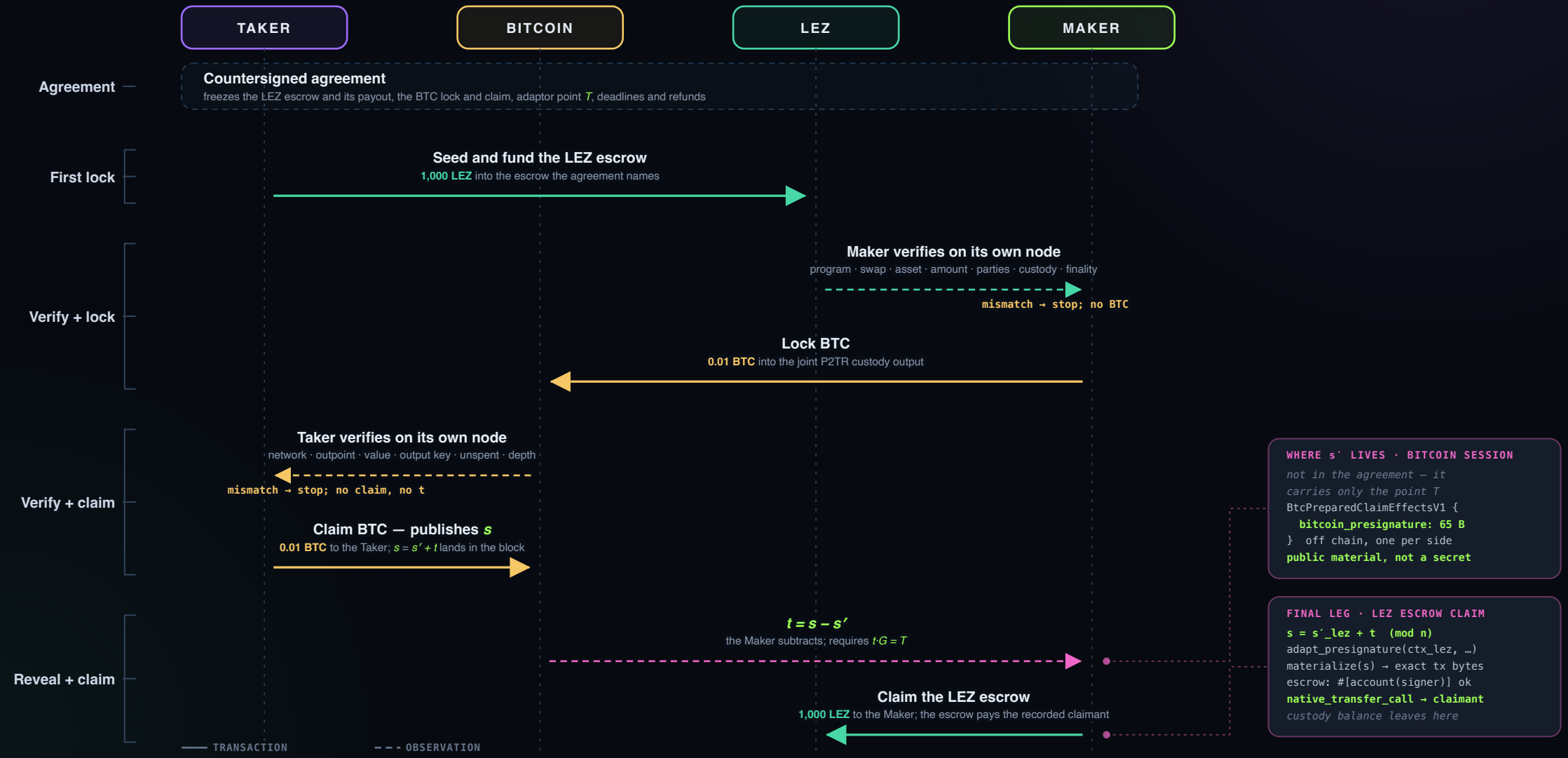
FINAL LEG · BITCOIN KEY PATH

```
s = s′_btc + t  (mod n)
adapt_presignature(ctx_btc, …)
verify_schnorr(s, sighash, output_key)
witness = p2tr_key_spend(s)
64 B on chain · no script, no branch
```

Threats TM-0005, TM-0012, TM-0016 · ADRs 0030, 0037, 0043, 0050

M3 · SWAP ALGORITHM · DIRECTION B · BTC → LEZ

Taker sells LEZ



Direction switch · `btc-swap-sdk/src/sdk.rs:3161` · funded chain · `swap-core/src/lib.rs:867-879`

M3 · SWAP ALGORITHM

What the direction decides

btc-swap-sdk/src/sdk.rs:3161 — which chain reveals

```rust
const fn claim_order(direction: SwapDirection) -> ClaimOrder {
    match direction {
        SwapDirection::TakerSellsForeign => ClaimOrder::LEZ_THEN_FOREIGN,
        SwapDirection::TakerSellsLez     => ClaimOrder::FOREIGN_THEN_LEZ,
    }
}
```

swap-core/src/lib.rs:867 — which party funds which chain

```rust
pub const fn funded_chain(&self, participant: Participant) -> Chain {
    match (self.direction, participant) {
        (TakerSellsForeign, Taker) | (TakerSellsLez, Maker) => foreign,
        (TakerSellsForeign, Maker) | (TakerSellsLez, Taker) => Chain::Lez,
    }
}
```

	A · TAKER SELLS BTC · LEZ → BTC	B · TAKER SELLS LEZ · BTC → LEZ
Taker funds	Bitcoin	LEZ
Maker funds	LEZ	Bitcoin
Escrow depositor	Maker	Taker
Escrow claimant	Taker	Maker
Refunds earlier	LEZ · maker-funded	Bitcoin · maker-funded

A · reveals on LEZ — untweaked context

```
ctx = claim_adaptor_session_context(Lez)
P   = KeyAgg(P_maker, P_taker)               // no tweak
m   = lez_terms.claim_message_hash
s′  = lez_presignature      (65 B = R̃ ‖ s′)
s   = observed LEZ claim signature (64 B = R̃ ‖ s)
t   = s − s′  (mod n)            // reveal_secret
      check t·G = T,  then adapt bitcoin_presignature
```

B · reveals on Bitcoin — BIP-341 tweaked context

```
ctx = claim_adaptor_session_context(Bitcoin)
P   = KeyAgg(P_maker, P_taker) + H_TapTweak(P, merkle_root)·G
m   = BIP-341 key-spend sighash
s′  = bitcoin_presignature  (65 B = R̃ ‖ s′)
s   = observed BTC claim signature (64 B = R̃ ‖ s)
t   = s − s′  (mod n)            // reveal_secret
      check t·G = T,  then adapt lez_presignature
```

Arrows are maker-side — what the offer pays out, into what it takes in. **The subtraction is identical; the context is not:** Bitcoin aggregates under a BIP-341 tweaked output key and records its parity, LEZ under the plain aggregate. **The invariant holds in both:** the maker-funded leg always carries the earlier deadline, so the party that moved first is the last able to recover.

Direction · `btc-swap-sdk/src/sdk.rs:3161` · funded chain · `swap-core/src/lib.rs:867-879`

M3 · LEZ ESCROW DESIGN

Creation

AGREEMENT COUNTERSIGNED → TAKER LOCKS BTC → **MAKER SEEDS THE LEZ ESCROW** → TAKER VERIFIES LEZ → TAKER CLAIMS LEZ → MAKER CLAIMS BTC

1 · adaptor-signature/src/lib.rs — the pre-signature is built

```rust
pub fn presignature(&self) -> Result<[u8; 65]> {
    let partials = match self.role {
        SigningRole::Maker => [own, peer],  SigningRole::Taker => [peer, own],
    };
    let adaptor_point = self.context.adaptor_point_value()?;      (1)
    musig2::adaptor::aggregate_partial_signatures(
        &self.context.key_aggregation, &self.aggregate_nonce()?,
        adaptor_point, partials, self.context.message)                (2)
}
```

2 · btc-swap-sdk/src/sdk.rs — two of them, and neither is trusted

```rust
bitcoin_presignature: [u8; 65],
lez_presignature:     [u8; 65],                                   (3)

fn validate(&self, agreement: &BtcAgreementV1) -> Result<(), BtcSdkError> {
    let bitcoin = agreement.claim_adaptor_session_context(Bitcoin)?;
    verify_adaptor_presignature(&bitcoin, self.bitcoin_presignature)?;    (4)
    let lez = agreement.claim_adaptor_session_context(Lez)?;
    verify_adaptor_presignature(&lez, self.lez_presignature)
}
```

3 · escrow/src/lib.rs — only now is the escrow created

```rust
pub fn initialize_native_witnessed(ctx,
    #[account(init, pda = arg("swap_id"))] metadata,
    #[account(mut, pda = [literal("custody"), arg("swap_id")])] custody,
    #[account(signer)] depositor, claimant, aggregate_authority, …)     (5)
{
    ClaimAuthority::AggregateWitness {
        x_only_public_key: aggregate_x_only_public_key,                (6)
        account_id: aggregate_authority.account_id }
}
```

1 The adaptor point enters the aggregation
The adaptor point from the countersigned agreement enters the aggregation. This is the moment the signature becomes deliberately short by exactly *t*.

2 65 bytes, and not a valid signature
Both partials combine into a pre-signature: bound to its message, its nonce and the joint key, and rejected by the chain as-is. Adding *t* is the only completion.

3 One pre-signature per chain
Same *t*, two messages. `s′_lez` is armed for the Taker; `s′_btc` is armed for the Maker. Neither can fire without the scalar.

4 Each verified against the agreement session
The session is rebuilt from the agreement commitment, so a pre-signature can only be accepted for the swap whose terms both roles signed.

5 Funding comes after arming
This is the ordering that matters: the Maker will not seed the escrow until it holds a verified `s′_btc`. The escrow is funded only after both pre-signatures verify.

6 The escrow records only the joint key
It records the joint key. Not *T*, not `s′`, not the message. Its only rule is that the joint account must sign.

Pre-signatures first, funding second.
Both pre-signatures exist and verify before either chain moves.

`s′_lez` armed for the Taker — completing it emits *t*

+

`s′_btc` armed for the Maker — completing it consumes *t*

→

escrow created — the money arrives last, into a mechanism already loaded

Pre-signature · `adaptor-signature/src/lib.rs:575, 734` · verification `btc-swap-sdk/src/sdk.rs:279–313` · escrow `:824–860`

M3 · LEZ ESCROW DESIGN

Happy path

AGREEMENT COUNTERSIGNED → TAKER LOCKS BTC → MAKER SEEDS THE LEZ ESCROW → TAKER VERIFIES LEZ → **TAKER CLAIMS LEZ** → MAKER CLAIMS BTC

1 · escrow/src/lib.rs — what the escrow demands, and where the LEZ moves

```
pub fn claim_native_witnessed(_ctx, metadata, custody,
    #[account(mut)] claimant,
    #[account(signer)] aggregate_authority: AccountWithMetadata, …)   (1)
let transfer = native_transfer_call(state.asset_program, custody.clone(),
        claimant.clone(), state.amount, true, &swap_id);   // ← the money   (8)
Ok(SpelOutput::execute(vec![metadata, custody, claimant, aggregate_authority],
                       vec![transfer]))
```

2 · btc-swap-sdk/src/sdk.rs — the Taker spends the LEZ leg (ClaimLeg::Lez)

```
let context   = agreement.claim_adaptor_session_context(Lez)?;
let signature = adapt_presignature(&context,
        prepared.claim_effects().lez_presignature,
        Zeroizing::new(*material.adaptor_secret))?;      // s = s′ + t   (2)
lez_claim.materialize(signature)?      // s becomes the tx signature   (3)
```

3 · btc-swap-sdk/src/sdk.rs — the Maker reads t back out (validate_lez_revealing_claim)

```
let context = agreement.claim_adaptor_session_context(Lez)?;
verify_final_signature(&context, observed.signature)?;              (4)
extract_adaptor_secret(&context,
    prepared.claim_effects().lez_presignature,
    observed.signature)                              // t = s − s′   (5)
```

4 · btc-swap-sdk/src/sdk.rs — and spends the Bitcoin leg with it (ClaimLeg::Foreign)

```
let context   = agreement.claim_adaptor_session_context(Bitcoin)?;
let signature = adapt_presignature(&context,
        prepared.claim_effects().bitcoin_presignature,
        Zeroizing::new(*material.adaptor_secret))?;   // the same t   (6)
let transaction = agreement.cooperative_claim().clone()
        .finalize(signature)?;               // the fixed BTC payout   (7)
```

1 The escrow takes no signature parameter
It takes no signature argument and no secret. The reveal follows from what the Taker must build to satisfy that one attribute.

2 First use of *t*
`adapt_presignature` adds the scalar to `s′_lez`. This is the only place the Taker's copy of *t* is consumed, and it happens locally.

3 *s* enters the claim transaction
`materialize` puts `s` into the LEZ claim transaction. Submitting it is what pays the Taker — and what publishes the completed signature.

4 Verification precedes subtraction
It rebuilds the same session from the agreement and checks the observed signature is genuinely valid, so it never subtracts against noise.

5 *t* recovered by subtraction
`extract_adaptor_secret` subtracts the `s′_lez` it has held since the signing rounds. Same nonce in both, so the difference is exactly the scalar.

6 The same *t* completes `s′_btc`
Now it completes `s′_btc` — the pre-signature armed for the Bitcoin leg back at creation, against the same *T*.

8 Where the LEZ leaves custody
`materialize` only splices *s* into a template. The balance leaves custody here, when the sequencer executes the instruction and the escrow emits `native_transfer_call` from the vault to the claimant.

7 The payout transaction is fixed
`cooperative_claim()` is the exact payout fixed in the agreement. The signature completes it and cannot redirect it.

One scalar, both legs.
The Taker's payment is the Maker's authorisation.

`s′_lez + t → s`
Taker paid in LEZ
`adapt_presignature`

→

`t = s − s′_lez`
Maker subtracts
`extract_adaptor_secret`

→

`s′_btc + t → s`
Maker paid in BTC
`adapt_presignature`

Claim `escrow/src/lib.rs:942–973` · adapt/extract `btc-swap-sdk/src/sdk.rs:2686–2698, 2762–2792`

M3 · LEZ ESCROW DESIGN

Permissionless methods

AGREEMENT COUNTERSIGNED → ONE OR BOTH LEGS LOCKED → NO REVEAL → DEADLINE PASSES → **REFUND**

escrow/src/lib.rs · refund_native — the whole instruction, as an outsider would call it

```
pub fn refund_native(_ctx: ProgramContext,
    #[account(mut, owner = self_program_id, pda = arg("swap_id"))] metadata, (1)
    #[account(mut, pda = [literal("custody"), arg("swap_id")])] custody,
    #[account(mut)] depositor: AccountWithMetadata,    // ← no signer    (2)
    swap_id: [u8; 32]) -> SpelResult
{
    let mut state = read_metadata(&metadata)?;
    require_funded(&state)?;
    if state.swap_id != swap_id
        || state.asset_program != state.custody_program
        || state.depositor != depositor.account_id                        (3)
        || state.custody != custody.account_id
        || custody.account.balance != state.amount
    { return Err(custom_error(ERROR_ACCOUNT_BINDING,
        "native refund account binding mismatch")); }

    let transfer = native_transfer_call(state.asset_program, custody.clone(),
        depositor.clone(), state.amount, true, &swap_id);
    state.status = EscrowStatus::Refunded;
    Ok(SpelOutput::execute(…).with_timestamp_validity_window(state.refund_at..)) (4)
}
```

(1) Addresses derive from the swap id

Both addresses derive from the swap id, so a watchtower computes them itself — no API key, no index, no cooperation from either party.

(2) No signer on the refund

The only instruction in the program without `#[account(signer)]`. Anyone holding the swap id can submit it.

(3) The depositor binding holds

Permissionless is not unrestricted. A stranger who submits it pays the fee and moves nobody else's money.

(4) Valid only after `refund_at`

Consensus will not include it earlier, so an outsider cannot pre-empt a live swap. The claim stays closed to outsiders — it needs the joint signature.

(5) Outage cases

Taker offline: anyone recovers its deposit for it. Maker daemon down: the same. Neither outage strands funds, because the recovering transaction needs nothing either party holds.

Refund instruction · `escrow/src/lib.rs:977–1021` · claim signer `:949`

M3 · LEZ ESCROW DESIGN

Refunds

AGREEMENT COUNTERSIGNED → ONE OR BOTH LEGS LOCKED → NO REVEAL → DEADLINE PASSES → **REFUND**

the two refund mechanisms, one per chain

```
// LEZ — the windows tile the timeline; consensus enforces both ends.
Ok(SpelOutput::execute(…).with_timestamp_validity_window(..state.refund_at)) // claim
Ok(SpelOutput::execute(…).with_timestamp_validity_window(state.refund_at..)) // refund (1)

// The whole recorded amount, back to the recorded depositor.
let transfer = native_transfer_call(state.asset_program, custody.clone(),
    depositor.clone(), state.amount, true, &swap_id);                (2)
state.status = EscrowStatus::Refunded;                                (3)

// refund_at is fixed at initialize and never rewritten.
pub refund_at: u64,      // EscrowMetadata field
if … || refund_at == 0 { return Err(ERROR_INVALID_TERMS) }            (4)

// ── Bitcoin, btc-swap-sdk/src/p2tr.rs — script path of the same output
let refund_script = Builder::new()
    .push_sequence(refund_delay.bitcoin_sequence())
    .push_opcode(OP_CSV).push_opcode(OP_DROP)                         (5)
    .push_x_only_key(&refund_key.0).push_opcode(OP_CHECKSIG)
    .into_script();                                                   (6)
```

1 Disjoint, exhaustive windows

`..refund_at` and `refund_at..` leave no overlap and no gap, so the LEZ side has no claim-versus-refund race at all.

2 Full amount to the depositor

No partial refunds exist. Exactly what was recorded goes back to exactly the account that funded it.

3 One-way status change

Replaying the instruction is not a second refund.

4 `refund_at` is fixed at creation

Validated non-zero at creation and never rewritten. Neither party can extend or shorten it afterwards.

5 `OP_CSV` counts blocks

`OP_CSV` counts blocks from the funding output, not wall-clock. Different mechanism, same job.

6 The funder's key signs alone

The funder's own refund key signs alone. Revealing this leaf also reveals the control block, which is why the cooperative path never touches it.

LEZ refund · `escrow/src/lib.rs:973, 1021` · Bitcoin leaf · `btc-swap-sdk/src/p2tr.rs:234–240`

M3 · BITCOIN CUSTODY DESIGN

Creation

AGREEMENT COUNTERSIGNED → **TAKER LOCKS BTC** → MAKER SEEDS THE LEZ ESCROW → TAKER VERIFIES LEZ → TAKER CLAIMS LEZ → MAKER CLAIMS BTC

1 · btc-swap-sdk/src/p2tr.rs — the refund branch, then the output

```
let refund_script = Builder::new()
    .push_sequence(refund_delay.bitcoin_sequence()).push_opcode(OP_CSV)
    .push_opcode(OP_DROP).push_x_only_key(&refund_key.0).push_opcode(OP_CHECKSIG)
    .into_script();                                                    (1)
let spend_info = TaprootBuilder::new().add_leaf(0, refund_script.clone())?
    .finalize(&secp, UntweakedPublicKey::from(aggregate_key.0))?;      (2)
if !control_block.verify_taproot_commitment(&secp, output_key…, &refund_script)
    { return Err(P2trSwapOutputError::InvalidControlBlock); }          (3)
let script_pubkey = ScriptBuf::new_p2tr_tweaked(output_key);
```

2 · btc-swap-sdk/src/transaction.rs — the exact payout, and the message it commits to

```
let unsigned_transaction = Transaction { version: TWO, lock_time: ZERO,
    input:  vec![TxIn { previous_output: funding_outpoint, … }],
    output: outputs };                  // fixed by the agreement   (4)
let sighash = SighashCache::new(&unsigned_transaction)
    .taproot_key_spend_signature_hash(0, &Prevouts::All(&prevouts),
        TapSighashType::Default)?;                                    (5)
```

3 · btc-swap-sdk/src/sdk.rs — and the pre-signature armed against that message

```
let bitcoin = agreement.claim_adaptor_session_context(Bitcoin)?;
verify_adaptor_presignature(&bitcoin, self.bitcoin_presignature)?;    (6)
// s'_btc: 65 bytes, valid pre-signature, invalid signature
```

(1) The only script in the output

Wait *n* blocks, then one signature from the funder's key. Nothing about swaps, secrets or the other chain appears in it.

(2) One leaf on the joint internal key

The aggregate MuSig2 key is the internal key; the refund leaf is the only branch. Cooperative spends never reveal that the branch exists.

(3) Construction fails on a bad commitment

If the control block does not verify against the output key and script, construction fails rather than producing an unspendable address.

(4) Inputs and outputs fixed at construction

Inputs and outputs are set at construction. Whatever signature arrives can only authorise *this* transaction.

(5) The key-spend sighash is the signed message

The BIP-341 key-spend sighash is the message the adaptor session signs — the Bitcoin twin of the LEZ `claim_message_hash`.

(6) `s'_btc` verified before use

Same shape as the LEZ side: a 65-byte pre-signature, checked against a session rebuilt from the agreement, short by exactly *t*.

Output · `btc-swap-sdk/src/p2tr.rs:229-277` · payout and sighash · `transaction.rs:129-165`

M3 · BITCOIN CUSTODY DESIGN

Happy path

AGREEMENT COUNTERSIGNED → TAKER LOCKS BTC → MAKER SEEDS THE LEZ ESCROW → TAKER VERIFIES LEZ → TAKER CLAIMS LEZ → **MAKER CLAIMS BTC**

1 · btc-swap-sdk/src/sdk.rs — the Maker completes the pre-signature with the recovered t

```rust
let signature = adapt_presignature(&context,
    prepared.claim_effects().bitcoin_presignature,
    Zeroizing::new(*material.adaptor_secret))?;        // s = s′ + t  (1)
```

2 · btc-swap-sdk/src/transaction.rs — the witness is attached, and the coin moves

```rust
pub fn finalize(mut self, signature_bytes: [u8; 64]) -> Result<Transaction> {
    Secp256k1::verification_only().verify_schnorr(&signature.signature,
        &message, &self.output_key.to_x_only_public_key())?;            (2)
    self.unsigned_transaction.input[0].witness =
        Witness::p2tr_key_spend(&signature);                            (3)
    Ok(self.unsigned_transaction)      // spendable: broadcast it       (4)
}
```

3 · where the value actually leaves the custody output

```
// input[0]  spends  funding_outpoint   — the joint P2TR output
// output[]  pays    the Maker's claim_destination_script_pubkey
// both were fixed in the countersigned agreement, before any lock   (5)
```

1 The *t* recovered from the LEZ claim
Recovered by subtraction moments earlier. The Bitcoin leg needs no new secret and no new round of messages.

2 Checked against the tweaked output key
`finalize` refuses a signature that does not verify for the exact sighash and the exact key path. A wrong scalar cannot be broadcast by accident.

3 Single-key witness
One 64-byte Schnorr signature. No script, no control block, no branch — indistinguishable on chain from an ordinary Taproot payment.

4 Relayable bytes
From here it is bytes anyone can relay. Broadcasting it is what moves the coin; the SDK does not custody anything.

5 Destination fixed in the agreement
The output script came from `claim_destination_script_pubkey` in the signed body. The signature completes the payout; it cannot redirect it.

The same scalar completes both legs.
The LEZ claim published *t*; the Bitcoin claim spends it.

`t = s − s′_lez` — recovered from the LEZ claim signature → `s′_btc + t → s` — **adapt_presignature** then **finalize** → **0.01 BTC → Maker** — key-path spend of the joint custody output

Adapt · `btc-swap-sdk/src/sdk.rs:2762-2778` · witness · `transaction.rs:210-227`

M3 · BITCOIN CUSTODY DESIGN

Permissionless methods

AGREEMENT COUNTERSIGNED → ONE OR BOTH LEGS LOCKED → NO REVEAL → **CSV MATURES** → REFUND

1 · btc-swap-sdk/src/transaction.rs — finalize returns a plain Transaction

```rust
pub fn finalize(mut self, signature_bytes: [u8; 64]) -> Result<Transaction>
// …no node handle, no wallet, no broadcaster. Just consensus-valid bytes.  (1)
```

2 · the refund transaction can be built and signed long before it is usable

```rust
let refund = RefundScriptPathSpend::new(&contract, funding_outpoint,
        funding_value, outputs)?;
let signed = refund.finalize(funder_signature)?;   // valid bytes, held  (2)
// input[0].sequence = CSV delay ⇒ relay refuses it until maturity     (3)
```

3 · what an outsider can and cannot do

```rust
// CAN : relay either signed transaction — anyone, any node        (4)
// CANNOT: alter destination or amount — both are inside the sighash
// CANNOT: produce a signature — needs the joint key, or the funder's  (5)
```

1 Broadcast requires no permission

Bitcoin has no notion of a submitter. A fully signed transaction is public bytes that any node will relay, so no party has to be online to publish one.

2 The refund can be signed in advance

Built and signed at agreement time, then handed to a watchtower or retained. It is invalid until the delay elapses.

3 `sequence` gates it at consensus

`sequence` carries the CSV delay, so the network itself rejects the refund early. Nobody has to enforce the deadline in software.

4 Outage cases

Funder offline at maturity: whoever holds the signed refund relays it. Maker offline after the reveal: the claim it already signed can be relayed by anyone.

5 No new authorisation is possible

An outsider can only publish what was already signed. Redirecting either spend needs a signature over a different sighash, which needs a key they do not have.

Spend construction · `btc-swap-sdk/src/transaction.rs:112-227, 287-414`

M3 · BITCOIN CUSTODY DESIGN

Refunds

AGREEMENT COUNTERSIGNED → ONE OR BOTH LEGS LOCKED → NO REVEAL → CSV MATURES → **REFUND**

1 · btc-swap-sdk/src/p2tr.rs — the leaf that makes it possible

```
let refund_script = Builder::new()
    .push_sequence(refund_delay.bitcoin_sequence())      // relative, in blocks (1)
    .push_opcode(OP_CSV).push_opcode(OP_DROP)
    .push_x_only_key(&refund_key.0).push_opcode(OP_CHECKSIG);             (2)
```

2 · btc-swap-sdk/src/transaction.rs — the spend commits to the delay and the leaf

```
input: vec![TxIn { previous_output: funding_outpoint,
    sequence: contract.refund_delay().bitcoin_sequence(), … }],          (3)
let sighash = SighashCache::new(&unsigned_transaction)
    .taproot_script_spend_signature_hash(0, &Prevouts::All(&prevouts),
        contract.tapleaf_hash(), TapSighashType::Default)?;              (4)
```

3 · btc-swap-sdk/src/transaction.rs — and the witness reveals the branch

```
Secp256k1::verification_only()
    .verify_schnorr(&signature.signature, &message, &self.refund_key)?;
self.unsigned_transaction.input[0].witness = Witness::from_slice(&[
    signature_bytes.as_slice(), self.refund_script.as_bytes(),
    self.refund_control_block.as_slice() ]);                             (5)
```

1 Relative delay, counted in blocks

`OP_CSV` counts blocks from the funding output's confirmation. It cannot be gamed by clock skew, and it starts only once the lock is real.

2 The funder's key alone

The funder's own refund key, alone. No joint signature, no counterparty, nothing to negotiate at the point of recovery.

3 `sequence` must match the leaf

The script checks it, and the input's `sequence` asserts it. Both must agree or the spend is invalid — the delay cannot be shortened by rebuilding the transaction.

4 Script-path digest differs

Script-path spends commit to the tapleaf hash, so a key-path signature can never satisfy this branch and vice versa.

5 The branch is revealed only on use

The witness carries signature, script and control block. Until a refund happens, nobody watching the chain learns the output had a refund path at all.

Refund leaf · `btc-swap-sdk/src/p2tr.rs:234-240` · script-path spend · `transaction.rs:309-414`

M3

Four ways a swap ends.

A

No first lock

Either party stops.

No funds exposed

B

Only first lock

After signed cutoff, two fresh absence reads, and a fresh unspent eligibility check:

Taker refunds first leg

C

Both locks, no reveal

Second leg refunds first. Only after that exact refund is canonical:

First leg refunds later

D

Reveal published

Maker verifies and extracts, then must obtain the follow-up claim before the later boundary.

Liveness window matters

Important: after Bitcoin CSV maturity, key-path claim and refund may compete for the same outpoint. The safety margin keeps the normal flow away from that race; this is not a cross-chain ACID claim.

Refund evidence · `m3-local-two-direction-refund-poc-20260716.json`

What a quantum computer breaks.

PRIMITIVE	ASSUMPTION	UNDER A CRQC
Adaptor relation $T = t \cdot G$	secp256k1 discrete log	Shor — this is the atomicity
BIP-340 Schnorr	secp256k1 · both chains	Shor
MuSig2 custody key	secp256k1 aggregate	Shor
AccountId derivation	SHA-256 of x-only key	Grover only — 128-bit
Offer envelope digests	SHA-256	Grover only — 128-bit
LEZ execution proofs	RISC Zero STARK	Hash-based — holds

Bitcoin is not post-quantum either.

1. **Post-quantum atomicity would protect nothing here.**
   The same adversary spends the P2TR output directly — a Taproot output key is on chain from the moment it is funded.
2. **So the mechanism was chosen on properties we can actually have.**
   No script on either chain, no preimage on chain, nothing linking the two legs, one fewer output to spend.
3. **Upgrading is a mechanism swap, not a redesign.**
   `ClaimAuthority` is per escrow, and `Sha256Preimage` already runs the ZEC corridor. Agreement, deadlines and refunds do not depend on the signature scheme.

BTC leg · adaptor	discrete log
ZEC leg · BIP-199 hashlock	hash — already PQ-shaped

The gap is real: an HTLC's secret is a hash preimage — Grover only halves it, so its atomicity survives. Ours is a scalar with $T = t \cdot G$ published in the agreement, so Shor recovers t and claims both legs. Bitcoin gives us no way to close that today: lattice (LAS) and isogeny (IAS) adaptor signatures exist in research but need a soft fork. Exposure is real-time, not harvest-now — a settled swap cannot be stolen retroactively.

Not a claimed property · tracked as a design assumption, not a mitigation

M3

Five scenarios recorded, all passing.

SECRET-SAFE
PUBLIC RECORDS

SCENARIO	COVERAGE	RESULT	RECORD
Settlement	Both directions · actual BTC + LEZ nodes	PASS	`two-direction-poc`
Ordered refunds	Both directions · exact terminal replay	PASS	`two-direction-refund`
First-lock recovery	Absent second lock · both directions	PASS	`first-lock-refund`
Post-reveal restart	Fresh Maker-controlled survivor process	PASS	`survivor-claim`
Overlap	Two opposite-direction swaps in flight	PASS	`overlapping-two-swap`

RECORDED
public transaction + block identifiers

OMITTED
keys · credentials · adaptor scalar · raw private state

BOUNDARY
private-local functional evidence, not mainnet readiness

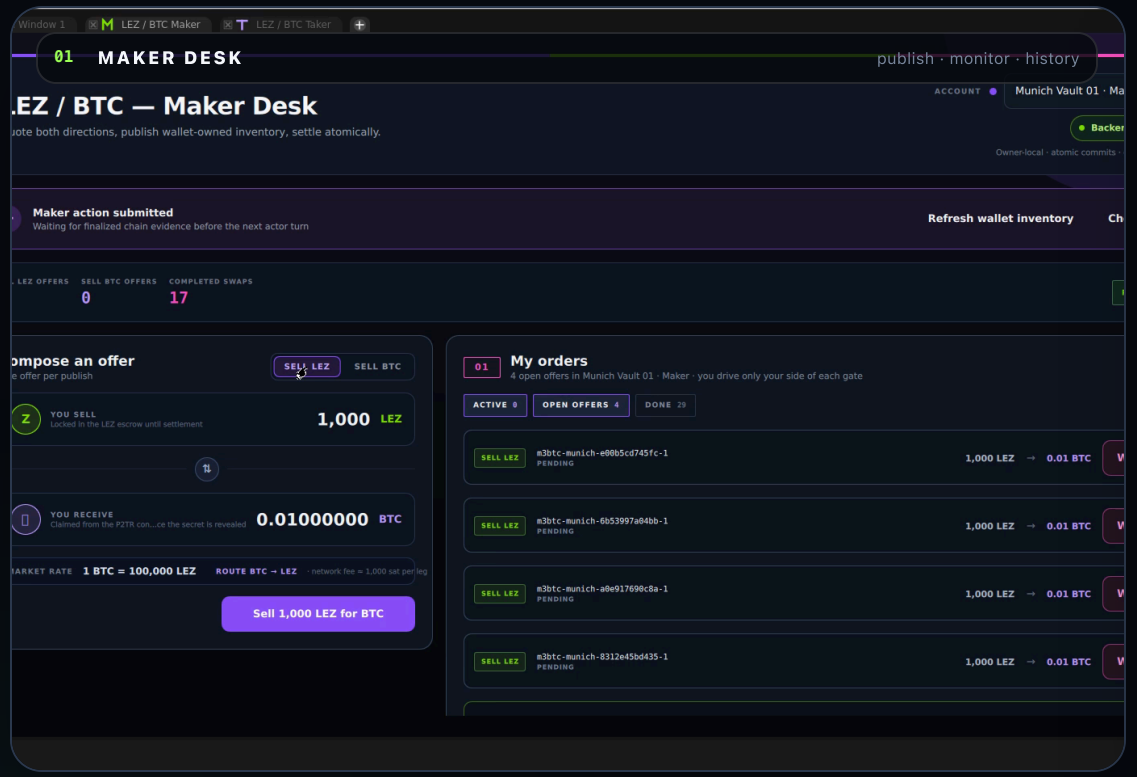
Index · `docs/milestone-3-review.md` + `docs/evidence/m3-*.json`

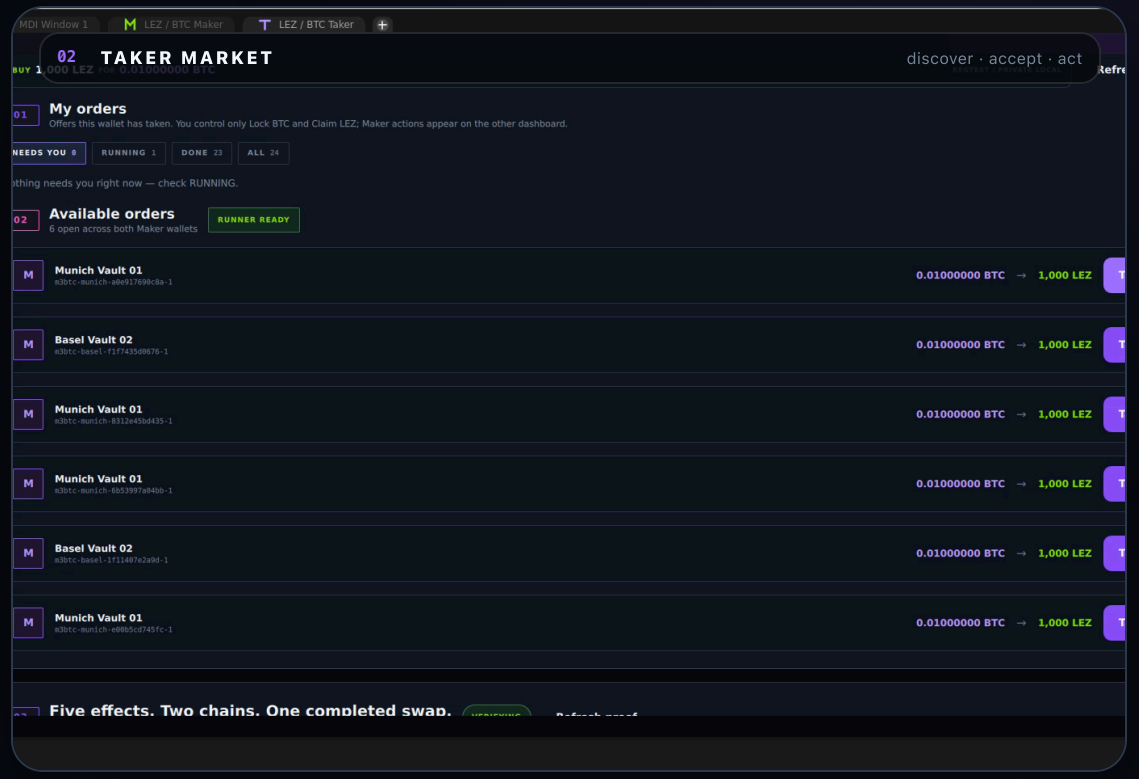
M6

Basecamp apps

M6

Each desk shows only its own action.





Taker · Lock BTC → Maker · Fund LEZ → Taker · Claim LEZ → Maker · Claim BTC

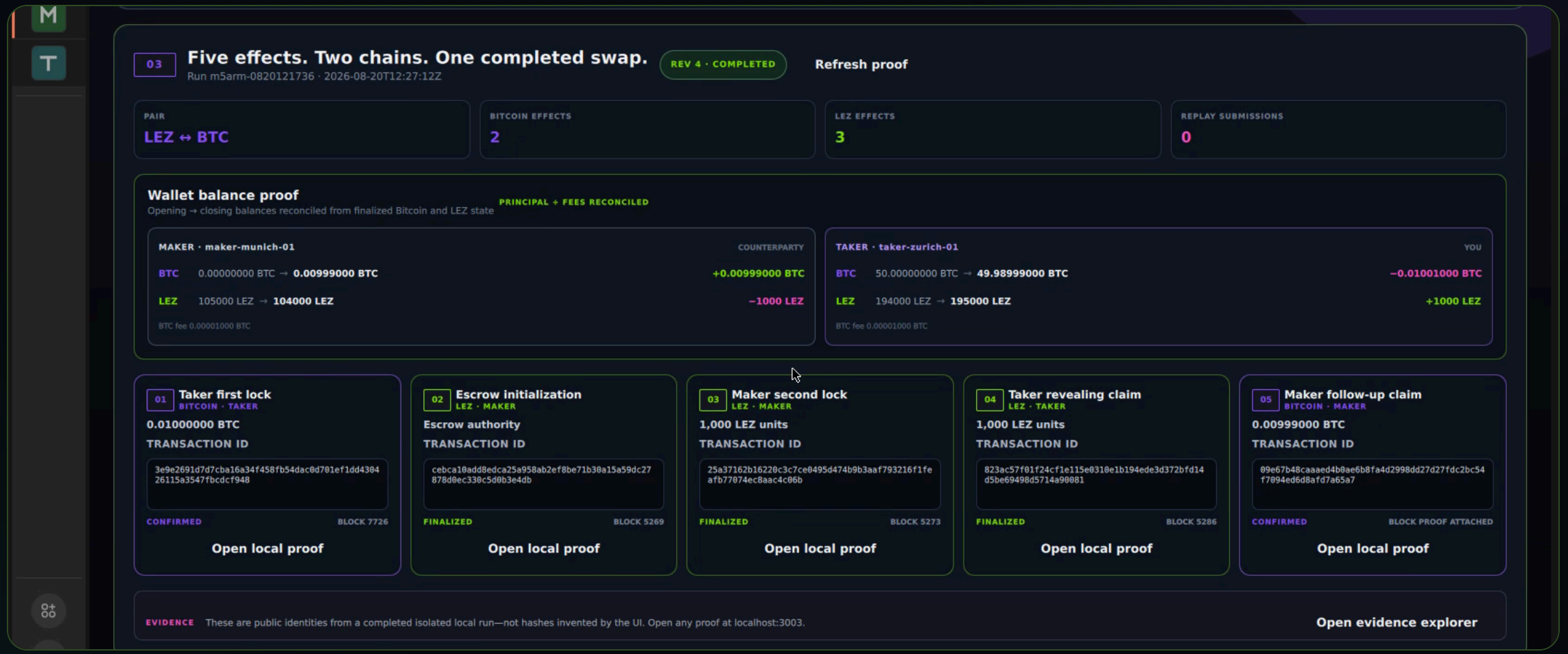
Current branch UI · `apps/basecamp/maker` + `apps/basecamp/taker`

M3 + M6

One completed swap, five on-chain actions.

RUN m5arm-0820121736

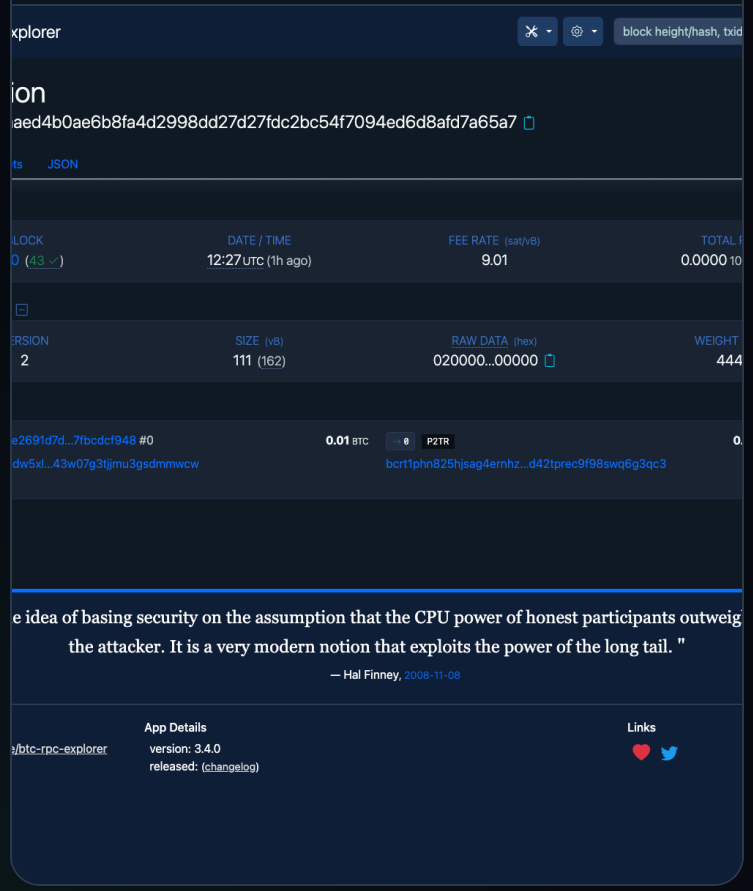
AMOUNT	ON-CHAIN ACTIONS ("EFFECTS" IN EVIDENCE)	STATE	REPLAY
0.01 BTC ⇄ 1,000 LEZ	2 Bitcoin + 3 LEZ	revision 4 · completed	0 duplicate submissions



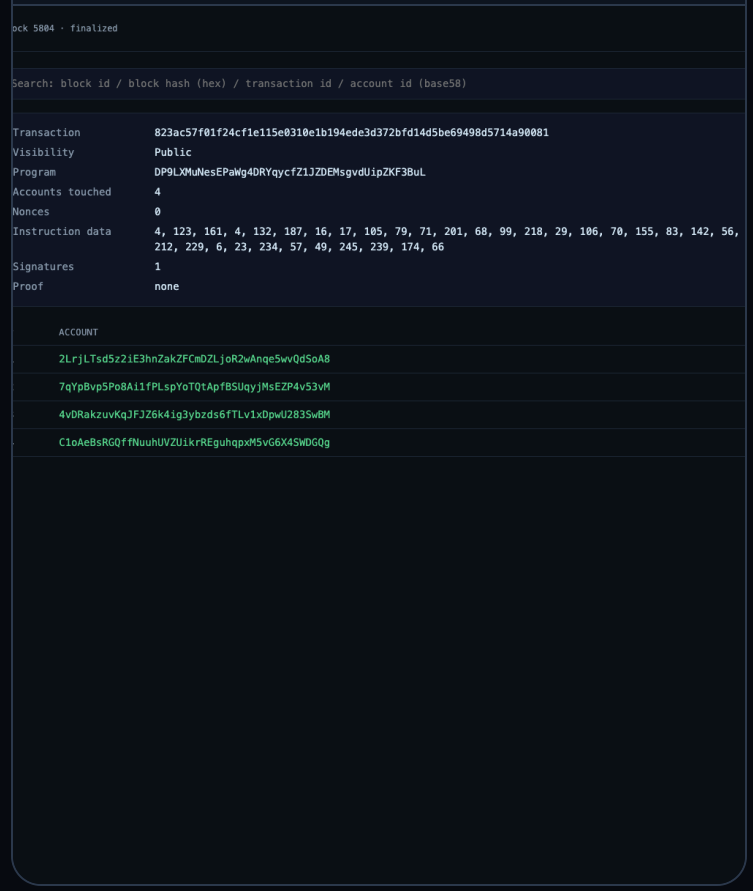
Machine-readable export · docs/evidence/m3-btc-ui-run-m5arm-0820121736.json

Every transaction resolves in both explorers.

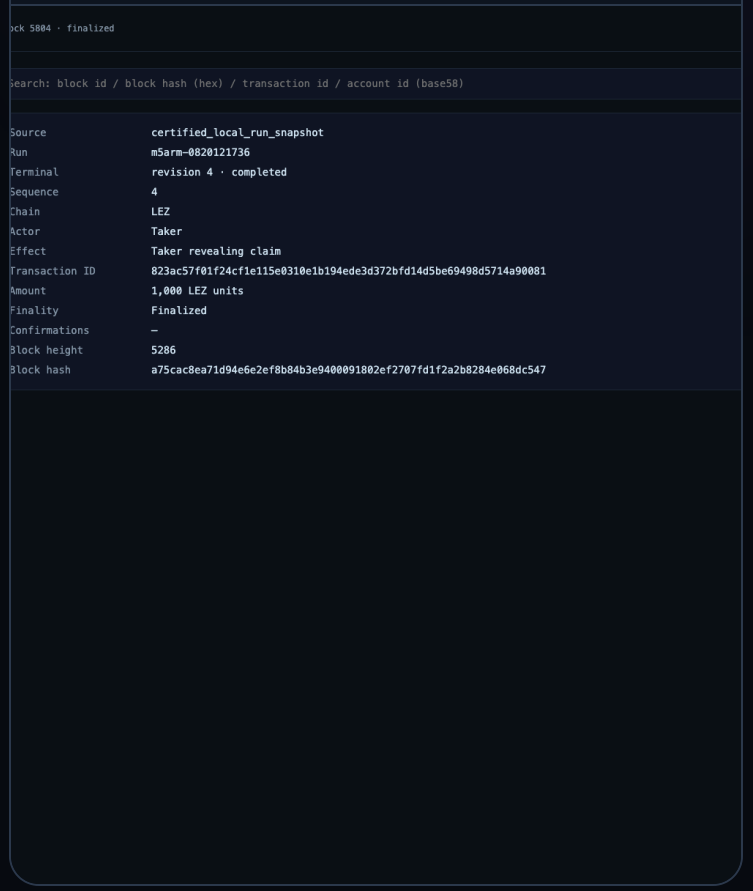
Bitcoin follow-up claim
P2TR · confirmed · exact outpoint spent



LEZ revealing claim
finalized · exact witnessed signature match



Evidence record
block + account identities · secret-safe



Public identities from disposable local chains · no keys, seeds, or adaptor scalar included

M6

Two separately installable role packages.

BASECAMP PACKAGE

Maker

Pair and price configuration, offer inventory, active swaps, history, role-local health.

BASECAMP PACKAGE

Taker

Offer browsing, exact-term consent, progress, mutually exclusive terminal action, proof.

6/6
isolated browser prototype journeys

2
separately installable role packages

13
package contract slots

1
owner-local transport boundary

The video proves the current BTC journey. Historical M6 evidence additionally reviews refund and ZEC privacy guidance without claiming those screens are shown here.

Review + certificate · `docs/m6-prototype-review.md` · `m6-basecamp-role-packages-20260804.json`

INCLUDED

Private-local functional proof

- ✓ M1 design, threat model, parameters, and SDK surface
- ✓ M3 both directions, refunds, survivor, and overlap records
- ✓ Current BTC happy path through Basecamp Maker/Taker UI
- ✓ Real local nodes, explorers, balance proof, replay checks

NOT CLAIMED

Production or unrelated milestones

- — Public-testnet or mainnet deployment
- — Production custody, formal audit, or M7 readiness
- — M2 Zcash, M4 Monero, or complete M5 delivery
- — Fee-stress, deep-reorg, or arbitrary boundary-race closure
- — Post-quantum resistance on either signing path

Bitcoin Core 31.1 regtest · LEZ v0.2 private devnet · no public funds · no private signing material published

Full nonclaims and provenance · `docs/submission.md`

How to review this.

01 Read the milestone map

02 Inspect public evidence

03 Run the private-local stack

04 Open both explorers

WATCH

`media/lez-btc-ui-swap-demo.mp4`

113 s · Basecamp UI · run m5arm-0825151914 · captions in .en.vtt

READ

`submission/README.md`

MILESTONES · EVIDENCE · REPRODUCE · LIMITATIONS

LEZ ⇄ BITCOIN · M1 + M3 + M6 SUBMISSION